Engineering Decisions

Risk Scoring Formula

The risk scoring formula is deterministic and weighted, not machine learning. Four component scores are computed - attendance (35%), practice questions (30%), trend direction (20%), and days since last facilitator note (15%) - and summed to produce a risk_score between 0 and 100. The score is then bucketed into four risk levels using configurable thresholds: CRITICAL (75+), HIGH (50 - 74), MEDIUM (25 - 49), LOW (below 25). All weights and thresholds are defined as constants in src/config.py and are env-var overridable.

The choice of a deterministic formula over a machine learning model was deliberate. ML models require labelled outcome data (which students who were intervened on actually re-engaged?) that Boon Academy does not yet have. Without outcome labels, a trained model would merely learn to replicate the same pattern-matching that an experienced facilitator already does intuitively - without any guarantee of being better calibrated. After six months of real intervention data with outcome tracking, revisiting an ML approach would be warranted.

The weights in the current formula (0.35 / 0.30 / 0.20 / 0.15) are starting assumptions, not empirically validated parameters. They represent the team's best current belief about which engagement signals most predict disengagement. These weights should be reviewed with the academic director before the pipeline is used on real students, and recalibrated annually as outcome data accumulates. The formula is deliberately transparent so that a facilitator who questions a student's risk score can trace exactly which component drove it.

LLM Batching Strategy

The pipeline makes one API call per campus, not one API call per student. In the demo run of 300 students across 20 campuses, this produced 14 API calls rather than the approximately 120 that a per-student approach would have required (300 students × 40% at-risk rate). Campus batching reduces API cost and latency by roughly an order of magnitude while also providing the model with cohort context - the model sees all CRITICAL and HIGH students on a campus together, which allows it to write messages that are appropriately calibrated relative to the cohort rather than in isolation.

The batching is bounded by MAX_STUDENTS_PER_LLM_CALL (default: 10). If a campus has more than 10 at-risk students, it is split into multiple calls. This prevents prompts from exceeding the model's context window and keeps response latency predictable. In practice, campuses have 15 students total and a 40% at-risk rate gives approximately 6 students per campus, well within the batch limit.

Tool use (structured JSON output via the Anthropic tools API) is used instead of free-form text generation. This eliminates the need to parse markdown-wrapped JSON from model responses and ensures that the facilitator_summary and whatsapp_message fields always have a predictable structure. If the tool-use call fails (API error, malformed response), the three-layer fallback activates transparently.

Fallback Logic

The LLM integration uses a three-layer fallback to ensure the pipeline never halts on an API failure. Layer 1 is the Anthropic SDK's built-in retry mechanism (max_retries=3), which handles transient network errors and rate limit responses automatically with exponential backoff. Layer 2 is a single re-prompt attempt on API errors that survive Layer 1 (APIConnectionError, RateLimitError, APIStatusError, APITimeoutError). Layer 3 is a rule-based template applied if Layer 2 also fails, or if the API response is malformed (KeyError, StopIteration during parse).

Template-generated messages are written to the same facilitator_summary and whatsapp_message columns as LLM-generated messages but are tagged with generated_by: template rather than generated_by: llm. This tagging allows facilitators and downstream analytics to distinguish AI-generated content from rule-based content without inspecting the message text itself. The templates produce grammatically correct, actionable messages - they are not placeholders or error strings.

The fallback design reflects a core product requirement: the facilitator's daily workflow must not be blocked by a cloud API outage. A facilitator who arrives in the morning expecting their intervention list must receive it even if the Anthropic API experienced a disruption overnight. Template messages are conservative (they do not attempt personalisation) but they are useful - they correctly identify the at-risk students and suggest the appropriate intervention action based on risk level.

Output Format Choices

The pipeline produces six output formats, each chosen for the specific workflow of the person who will consume it. The primary artefact is an Excel workbook (intervention_priority_list.xlsx) because facilitators at Boon Academy already use Excel daily and need to sort, filter, and annotate the list. openpyxl is used rather than xlsxwriter because openpyxl supports both writing and reading the generated files - the test suite opens every output file and asserts on cell values, which requires read capability.

The WhatsApp CSV (whatsapp_messages.csv) is encoded in UTF-8 with BOM (utf-8-sig) so that Excel opens it correctly without character corruption on Windows, where Arabic script in WhatsApp messages would otherwise display as mojibake. The HTML dashboard (dashboard.html) is fully self-contained - all CSS and JavaScript are inlined, and the student data is embedded as a JSON constant so the file works when opened directly from the filesystem via file:// without requiring a web server.

The Word report (intervention_report.docx) uses only built-in python-docx styles (Heading 1, Normal, Table Grid) and no OxmlElement manipulation, which is a known instability in python-docx 1.1.2. The run_log.json file uses json.dumps(default=str) to handle datetime and Path objects without TypeError - a pattern established in output_generator.py and carried forward to any future module that writes JSON.

Intentional Simplicity

Several infrastructure components that are standard in production software were deliberately excluded from this pipeline: Docker, an authentication server, a real-time dashboard, a message queue, and a scheduler. These exclusions are not oversights - they are reasoned decisions based on the actual operational context: one part-time engineer, 20 campuses, a daily batch frequency, and a single-machine deployment model.

Docker would add container orchestration complexity without providing meaningful isolation benefits on a single facilitator machine. A real-time dashboard would require a persistent server process, which adds operational overhead (keeping it running, monitoring for crashes, updating) that the team cannot currently absorb. A message queue (Celery, Redis) would be appropriate if the pipeline needed to process campuses in parallel at sub-minute latency - it does not. Daily batch at 60 seconds total runtime makes message queues over-engineered.

The design philosophy is: add infrastructure when scale demands it, not in anticipation of demands that may never arrive. The migration path from CSV to SQLite to PostgreSQL provides a clear upgrade trajectory. When the pipeline is running at 50+ campuses and the sequential API calls are visibly slow, adding concurrent.futures parallelism is the right next step. Until then, keeping the system simple enough for one engineer to fully understand and debug in under an hour is a feature, not a limitation.